

Project Report : Operations Research

Boubou Yasmine - Créténier Adrien - Sabri Ilyas | January 20, 2026

4.1 Questions

Question 1 : Three-index formulation (MILP)

The binary variable x_{ij}^k is used to track each vehicle individually. Let $\mathcal{I}$ be the set of orders, $V = \mathcal{I} \cup \{0\}$ the set of vertices, and K the set of vehicles.

Decision variables :

- $x_{ij}^k \in \{0, 1\}$: equals 1 if vehicle k travels along arc (i, j) .
- $a_i^k \geq 0$: arrival time of vehicle k at customer i .
- $q_i^k \geq 0$: cumulative load of vehicle k after serving customer i .

Objective function : Minimize the total fuel cost.

$$\min \sum_{k \in K} \sum_{i \in V} \sum_{j \in V, j \neq i} c_f^{\text{fuel}} \cdot \delta^M(i, j) \cdot x_{ij}^k \quad (1)$$

Constraints :

- Each customer $i \in \mathcal{I}$ is visited exactly once :

$$\sum_{k \in K} \sum_{j \in V, j \neq i} x_{ij}^k = 1 \quad \forall i \in \mathcal{I} \quad (2)$$

- Flow conservation for each vehicle k :

$$\sum_{j \in V, j \neq i} x_{ji}^k - \sum_{j \in V, j \neq i} x_{ij}^k = 0 \quad \forall i \in V, \forall k \in K \quad (3)$$

- Capacity and subtour elimination (MTZ) :

$$q_j^k \geq q_i^k + w_j - Q(1 - x_{ij}^k) \quad \forall i, j \in \mathcal{I}, i \neq j, \forall k \in K \quad (4)$$

- Time propagation and delivery time windows :

$$a_j^k \geq a_i^k + \ell_i + \tau_{ij} - M(1 - x_{ij}^k) \quad \forall i \in V, \forall j \in \mathcal{I}, \forall k \in K \quad (5)$$

$$t_i^{\min} \leq a_i^k \leq t_i^{\max} \quad \forall i \in \mathcal{I}, \forall k \in K \quad (6)$$

- Restricted depot usage :

$$\sum_{j \in \mathcal{I}} x_{0j}^k \leq 1 \quad \forall k \in K \quad (7)$$

Question 2 : Two-index formulation (MILP)

Decision variables :

- $x_{ij} \in \{0, 1\}$: equals 1 if arc (i, j) is used.
- $u_i \in \{1, \dots, |\mathcal{I}|\}$: visiting rank of customer i .
- q_i, a_i : cumulative load and arrival time at vertex i .

Objective function :

$$\min \sum_{i \in V} \sum_{j \in V, j \neq i} c_f^{\text{fuel}} \cdot \delta^M(i, j) \cdot x_{ij} \quad (8)$$

Constraints :

— Degree balance for each customer :

$$\sum_{j \in V} x_{ij} = 1 \quad \text{and} \quad \sum_{j \in V} x_{ji} = 1 \quad \forall i \in \mathcal{I} \quad (9)$$

— Rank-based subtour elimination (classical MTZ) :

$$u_i - u_j + |\mathcal{I}|x_{ij} \leq |\mathcal{I}| - 1 \quad \forall i, j \in \mathcal{I}, i \neq j \quad (10)$$

— Capacity handling (load-based MTZ) :

$$q_j \geq q_i + w_j - Q(1 - x_{ij}) \quad \forall i, j \in \mathcal{I}, i \neq j \quad (11)$$

— Time windows (big-M) :

$$a_j \geq a_i + \ell_i + \tau_{ij} - M(1 - x_{ij}) \quad \forall i \in V, \forall j \in \mathcal{I}, i \neq j \quad (12)$$

$$t_i^{\min} \leq a_i \leq t_i^{\max} \quad \forall i \in \mathcal{I} \quad (13)$$

— Flow balance at the depot :

$$\sum_{j \in \mathcal{I}} x_{0j} = \sum_{i \in \mathcal{I}} x_{i0} \quad (14)$$

Question 3 : Dynamic Programming (Single Route - TSPTW)

State space : A state is defined by (X, i) , where $X \subseteq S$ is the set of customers already visited and $i \in X$ is the last customer visited. The value $V(X, i)$ is the earliest arrival time at i .

Recurrence relation :

$$V(X, j) = \min_{i \in X \setminus \{j\}} \left\{ \max \left(V(X \setminus \{j\}, i) + \ell_i + \tau_{ij}, t_j^{\min} \right) \right\} \quad (15)$$

Subject to the feasibility conditions $V(X, j) \leq t_j^{\max}$ and $\sum_{k \in X} w_k \leq Q$.

Initialization :

$$V(\{i\}, i) = \max(\tau_{0,i}, t_i^{\min}) \quad \forall i \in S \quad (16)$$

Question 4 : Lower Bound on the Number of Vehicles

To estimate the minimum number of vehicles $K_{\min}$ required, independently of costs, capacity and time limitations are combined.

1. Capacity bound (LB_{cap}) :

$$LB_{cap} = \left\lceil \frac{\sum_{i \in \mathcal{I}} w_i}{Q} \right\rceil \quad (17)$$

2. Time bound (LB_{time}) : We build an incompatibility graph $G = (\mathcal{I}, E')$, in which an edge links i and j if no vehicle can serve them consecutively, in either order :

$$(i, j) \in E' \iff (t_i^{\min} + \ell_i + \tau_{ij} > t_j^{\max}) \text{ and } (t_j^{\min} + \ell_j + \tau_{ji} > t_i^{\max}) \quad (18)$$

The size of the maximum clique $\omega(G)$ of this graph gives $LB_{time} = \omega(G)$.

3. Final bound and algorithm : The bound is given by $K_{\min} \geq \max(LB_{cap}, LB_{time})$. The corresponding algorithm is :

1. Sum the weights w_i to compute LB_{cap} .
2. Build G by testing the time feasibility of each pair (i, j) .
3. Compute a large clique of G , exactly or with a greedy heuristic. The size of any clique is a lower bound on $\omega(G)$, and therefore remains a valid lower bound on $K_{\min}$.
4. Return the maximum of the two bounds.

4.2 Algorithm

The VRPTW is NP-hard, and its difficulty is further increased here by time-dependent travel times, a heterogeneous fleet and a non-additive diameter cost. To handle instances with several hundred orders, we use an ALNS (Adaptive Large Neighborhood Search) metaheuristic. Its adaptive nature comes from a learning mechanism : destroy and repair operators are selected with probabilities updated according to their past performance.

In addition, a combinatorial *polishing* step is performed with CP-SAT (OR-Tools) on a *set partitioning* model. From a *pool* of routes collected during the search, CP-SAT selects the minimum-cost combination covering each customer exactly once. This step recombines high-quality routes already encountered in order to improve the overall solution.

1 Preprocessing

Local projection and distances. The coordinates (φ_i, λ_i) are projected onto metric coordinates (x_i, y_i) , using the depot latitude φ_0 as reference. Two matrices are precomputed : the Manhattan distance $\delta^M(i, j)$ and the Euclidean distance $\delta^E(i, j)$. The diameter of a route is $\max_{i,j \in r} \delta^E(i, j)$ and is used to apply the penalty $c^{\text{radius}}(r)$. These matrices make the evaluation of route modifications fast.

Tabulated time profile $\gamma_f(t)$. For each family f , $\gamma_f(t)$ is defined by a Fourier series. It is tabulated on a grid with step `time_step`, then linearly interpolated to obtain $\gamma_f(t)$ at any time. This avoids repeated trigonometric computations while preserving a continuous evolution of traffic conditions.

2 Time feasibility evaluation of a route

The feasibility of a sequence $0 \rightarrow i_1 \rightarrow \dots \rightarrow i_k \rightarrow 0$ for a family f is checked by simulation. For each arc (u, v) , the travel time depends on the departure time t and is given by $\tau_f(u, v | t) = \left(\frac{\delta^M(u, v)}{s_f} + p_f \right) \gamma_f(t)$. The arrival time is then updated as $a_v = t + \tau_f(u, v | t)$, followed by the service start time $\max(t_v^{\min}, a_v)$, which must satisfy $\max(t_v^{\min}, a_v) \leq t_v^{\max}$. Departure to the next node is only possible after the delivery duration ℓ_v . This test is applied successively along the route.

3 Initial solution construction (*regret-k* insertion)

The initial solution is built by *regret-k* insertion. We start by creating a few routes with the customers farthest from the depot, in order to form geographical seeds. Then, for each unassigned order, we evaluate its best insertion and its k best alternatives ; the *regret* is the cost difference between these options and measures the “criticality” of the order. At each iteration, the order with the highest regret is inserted, which secures hard-to-place customers early, before the capacity constraints w_f^{capacity} and time windows $t_i^{\max}$ shrink the search space.

4 ALNS : destroy/repair, local search and acceptance

ALNS iteratively improves a current solution. At each iteration, a destroy operator removes q customers (q drawn at random, typically between 4 and 10), and a repair operator then reinserts them using *regret-k*. Four destroy operators are used : *Random* (diversification), *Worst* (removes the customers with the highest detour cost), *Related* (removes a spatially/temporally close group) and *Radius* (removes peripheral customers with a reduced q to act on the diameter).

After repair, an intensification phase applies local moves : *relocate*, *swap* and *2-opt** (exchange of suffixes between two routes). Route elimination is regularly attempted in order to save the fixed cost c_f^{rental} . An integrity check then enforces an exact partition of $\mathcal{I}$ (no duplicates, no missing customer). Acceptance follows a simulated annealing scheme : a worse solution is accepted with probability $\exp(-\Delta/T)$, while operator weights are adjusted according to their past gains. Routes from the best solutions feed a *pool* used for polishing.

```
ALNS(current, time_limit):
```

```
    best = current
```

```
    initialize operator weights and temperature T
```

```
    while time not exceeded:
```

```
        op = draw destroy operator using weights    # select a destroy operator
```

```

cand = copy(current)
removed = DESTROY(cand, op, q)    # remove q customers according to op
REPAIR(cand, removed)            # regret-k reinsertion
LOCAL_SEARCH(cand)               # relocate/swap/2-opt* + route elimination
FIX_PARTITION_IF_NEEDED(cand)    # avoid duplicate/missing customers
if ACCEPT(cand, current, T):
    current = cand
    update operator scores(op)
    if cost(cand) < cost(best): best = cand
cool down T
update weights occasionally
add routes(best) to pool
best = POLISH_SET_PARTITIONING(best, pool) # periodic, time-limited
return best

```

5 Route elimination (reducing the number of vehicles)

Since the rental cost c_f^{rental} is dominant, a route elimination procedure is systematically integrated into the search. Short routes are removed first, and their customers are then reinserted into the other routes. When reinsertion is feasible, saving the fixed cost reduces the total cost $c(\mathcal{R})$, before a local re-optimization by the ALNS.

6 Set partitioning polishing (CP-SAT on a route pool)

A route pool is built during the ALNS by keeping the best versions observed (for a given family and sequence). The polishing step then solves a set partitioning problem : each route in the pool is a candidate column, and a minimum-cost subset covering each customer exactly once is selected. Solved by CP-SAT under a short time limit, this subproblem efficiently recombines locally optimized routes observed in different solutions, often improving the overall solution.

7 Numerical results

To generate a solution for a given instance, we run our heuristic solver with the command `python solver.py -vehicles vehicles.csv -instance instance_XX.csv -out routes_XX.csv -time_limit T -seed S -starts K -time_step dt -polish_every E -polish_time Tp`, where T is the maximum running time, S the seed used to initialize the pseudo-random number generator, K the number of multi-starts used to build several initial solutions, dt the discretization step (in seconds) of $\gamma_f(t)$, E the number of ALNS iterations between two polishing steps, and T_p the time limit (in seconds) allotted to each polishing step. Table 1 summarizes the parameters chosen to obtain the lowest costs. **Remarks :** Shorter running times already yield low costs (€9,708 in 1500s for instance 10, €3,436 in 480s for instance 6, €1,350 in 180s for instance 03). The choice of the seed is arbitrary.

Instance	Time limit (s)	Seed	Starts	Δt	polish_every	polish_time	Coût total (€)
1	120	123	4	60	40	10	452.49
2	150	123	4	60	40	10	775.76
3	500	123	4	60	40	10	1330.30
4	700	123	3	60	40	10	2034.89
5	1000	123	3	60	40	10	2623.61
6	2000	123	3	120	40	10	3252.84
7	3000	123	3	200	40	10	4443.54
8	4000	123	3	240	40	10	6032.85
9	5000	123	3	300	40	10	7420.59
10	10000	123	3	360	40	10	8921.80

TABLE 1 – Paramètres utilisés et coûts obtenus pour chaque instance.

The total cost achieved is **€37,288.67**.